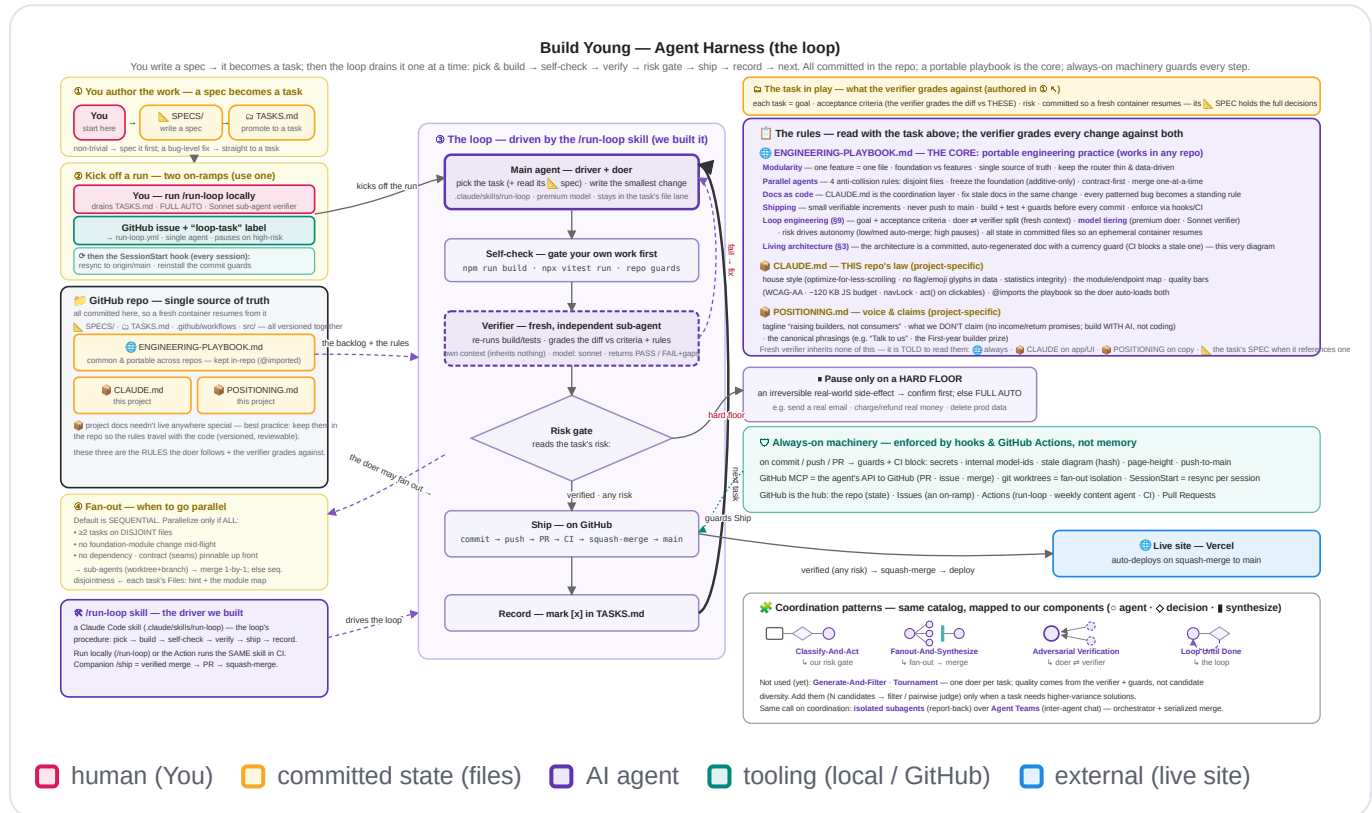


Build Young — Agent Harness (the loop)

You write a spec → it becomes a task; then the loop drains it one at a time — implement → self-check → verify → risk gate → ship → record → next. Everything's committed in the repo (a fresh container resumes); a portable playbook is the core; always-on machinery guards every step.



Detail — click a box above, or expand here

① You author the work — the whole process starts here

- You are the originator **human** — you write a one-page spec, then promote it to a task.
- Spec-first gate: non-trivial work → write a **spec first**; a bug-level fix → straight to a task.
- A spec (SPECs/NNN - name .md) pins what / why / behavior / **acceptance** / out-of-scope / risks. Promoted into TASKS.md ; BACKLOG.md is the parking lot.
- All committed in the repo **state** , so the loop — and a fresh container — resume from it.

▼ ② Kick off a run — two on-ramps that differ in autonomy

You — run `/run-loop` locally **human**

Your Claude subscription; drains the `TASKS.md` backlog. **FULL AUTO** — ships any task the moment the verifier **PASS**es, regardless of risk. Spawns a **separate Sonnet sub-agent** as the independent verifier (model tiering).

GitHub issue + “loop-task” label **cloud Action**

→ `run-loop.yml`; the issue IS the task. Runs as a **single premium agent** (can't spawn a sub-agent, so it does its **own** fresh-eyes verification) and **keeps the high-risk pause**: low/med → squash-merge + close the issue; high / architectural / destructive / ambiguous → open a PR, comment WHY, and STOP for a human.

Then the **SessionStart** hook (every session)

Resync to `origin/main` **only if the tree is clean** (never clobbers work) · reinstall the commit guards. A per-session mechanic, not an on-ramp.

▼ ③ The loop — pick → self-check → verify → risk gate → ship → record → next

- **Main agent — driver + doer** **premium**: picks the first task, **reads its** 📄 **spec**, writes the smallest change. The next step is a **signal** (failing test, verifier gap, next backlog item), never a guess.
- **Self-check**: `npm run build` · `npx vitest run` · repo guards. Fix until green.
- **Verifier — fresh, independent sub-agent** **sonnet**: own context, inherits nothing; re-runs build/tests and grades the diff vs the acceptance criteria + the standing rules it's told to read. **PASS** / **FAIL**+gaps; fail → fix → re-verify (~3 rounds). *(In the cloud issue Action there's no sub-agent — that single agent self-verifies.)*
- **Risk gate**: only the merge decision differs — local loop is **FULL AUTO** (any risk); the cloud issue Action pauses on high-risk (see Risk gate & hard floor).
- **Ship**: commit → push → PR → verify the file diff is non-empty → CI → **squash-merge** → **main** → sync.
- **Record**: mark `[x]` in `TASKS.md` → next task. Progress is committed, so an ephemeral container resumes.

▼ 📁 The repo — single source of truth

- `SPECS/` · `TASKS.md` · `.github/workflows` · `src/` — all versioned together.
- The standing **rules** live here too (playbook / **CLAUDE** / **POSITIONING**), so they travel with the code.
- Everything committed → a fresh/ephemeral container resumes exactly where it stopped.

▼ 🏠 **The task in play** — what the verifier grades against

- Each task = **goal** · **acceptance criteria** · **risk**, authored in ① and committed.
- The verifier grades the diff **vs the acceptance criteria** (+ the standing rules); its 🏠 SPEC holds the full decisions.

▼ 📋 **The rules** — the verifier grades every change against these

🌐 **ENGINEERING-PLAYBOOK.md** — the portable core (any repo)

Modularity · Parallel agents (4 anti-collision rules) · Docs-as-code · Shipping · **Loop engineering §9** (goal+acceptance · doer⇌verifier split · **model tiering**: premium doer · Sonnet verifier · spec-first gate is trigger-independent) · **Living architecture §3** (committed, auto-regenerated, currency-guarded).

🏠 **CLAUDE.md** — this repo's law

House style (optimize-for-less-scrolling · no flag/emoji glyphs in data · statistics integrity) · the module/endpoint map · quality bars (WCAG-AA · ~120 KB JS budget · navLock · act()).

🏠 **POSITIONING.md** — voice & claims

“raising builders, not consumers” · what we DON'T claim · canonical phrasings.

🔄 **update-playbook** — the rules self-improve **skill**

When a session establishes a durable, generalizable lesson (or a rule proves wrong), the agent proactively records it as principle + why — so the next change implements the learning. *The fresh verifier inherits none of this; the spawn prompt TELLS it what to read.*

▼ 🛡️ **Risk gate & hard floor** — autonomy differs by trigger

- **Local /run-loop = FULL AUTO**: any task auto squash-merges once the verifier PASSES, regardless of risk (overrides the playbook's “high-risk pauses” default).
- **Cloud issue Action = pauses on high-risk**: low/med auto-merge; high/architectural/destructive/ambiguous → PR + comment + stop for a human.
- **Both pause only on a HARD FLOOR**: an irreversible real-world side-effect → confirm first — e.g. **send a real email** · **charge/refund real money** · **delete prod data** · **rotate a live secret**. (Shipping code/copy to the repo is reversible — not a hard floor.)
- A “get a human's eyes eventually” note (e.g. Terms copy) → **flag-in-report**, don't block.

▼ ④ **Fan-out** — default sequential; parallelize only when provably safe

- Go parallel only if **ALL**: ≥ 2 tasks on **disjoint files** · **no foundation change** mid-flight · no dependency · the **contract is pinnable** up front.
- Then worktree-isolated sub-agents → **merge one-by-one**; after each merge the next rebases.
- **Parallelize the work, not the rigor** — each agent runs the full loop (its own verify).

▼ ✨ **The /run-loop skill (+ /ship)**

- `.claude/skills/run-loop` — the loop's procedure: pick → build → self-check → verify → ship → record. Run locally (`/run-loop`) or the Action runs the same contract in CI.
- **/ship** — the same verified path (build + tests + guards + independent verifier) as a standalone skill, so a **direct edit** merges with identical rigor — verification is a property of shipping, never an unverified shortcut.

▼ ⚙️ **Always-on machinery** — enforced by hooks & GitHub Actions, not memory

- **Commit guards** (installed into `.git/hooks` by SessionStart): hard-block an internal/bracketed model-id (in the diff **and** the commit message) + a **stale diagram** (hash mismatch); advisory on `\uXXXX` / money-sim markers.
- **CI** (path-scoped, blocks the merge): `architecture-current` (diagram in sync) · `landing-lean` (landing height ceiling). No blanket build/test run on every PR — that's the loop's self-check + verifier.
- `settings.json` : allowlist of safe commands + a hard **deny on git push ... main** .
- **GitHub MCP** = the agent's API to GitHub · **git worktrees** = fan-out isolation · **SessionStart** = resync per session.

▼ 🌐 **Live site — Vercel**

- A **verified** change (any risk) → squash-merge → **auto-deploy**. No manual release step.

▼ 🧩 **Coordination patterns** — the standard catalog, mapped to our components

- **Classify-And-Act** → our risk gate.
- **Fanout-And-Synthesize** → fan-out → merge.
- **Adversarial Verification** → doer $\nRightarrow$ verifier.
- **Loop Until Done** → the loop itself.
- **Not used (yet)**: Generate-And-Filter · Tournament — one doer per task; quality comes from the verifier + guards, not candidate diversity.

This page is the canonical loop diagram (the single source). Static exports are generated from it —
`docs/architecture/loop.png` (the diagram, embedded in BUILD-YOUNG-ARCHITECTURE.md) + `loop.pdf`
(the diagram + all detail, the handout) — via `bash scripts/render-architecture.sh`.